

C H A P T E R

18

TOOLS FOR LARGE PROGRAMS

CONTENTS

Section 18.1 Exception Handling	772
Section 18.2 Namespaces	785
Section 18.3 Multiple and Virtual Inheritance	802
Chapter Summary	816
Defined Terms	816

C++ is used on problems small enough to be solved by a single programmer after a few hours' work and on problems requiring enormous systems consisting of tens of millions of lines of code developed and modified by hundreds of programmers over many years. The facilities that we covered in the earlier parts of this book are equally useful across this range of programming problems.

The language includes some features that are most useful on systems that are more complicated than those that a small team can manage. These features—exception handling, namespaces, and multiple inheritance—are the topic of this chapter.

Large-scale programming places greater demands on programming languages than do the needs of systems that can be developed by small teams of programmers. Among the needs that distinguish large-scale applications are

- The ability to handle errors across independently developed subsystems
- The ability to use libraries developed more or less independently
- The ability to model more complicated application concepts

This chapter looks at three features in C++ that are aimed at these needs: exception handling, namespaces, and multiple inheritance.

18.1 Exception Handling

Exception handling allows independently developed parts of a program to communicate about and handle problems that arise at run time. Exceptions let us separate problem detection from problem resolution. One part of the program can detect a problem and can pass the job of resolving that problem to another part of the program. The detecting part need not know anything about the handling part, and vice versa.

In § 5.6 (p. 193) we introduced the basic concepts and mechanics of using exceptions. In this section we'll expand our coverage of these basics. Effective use of exception handling requires understanding what happens when an exception is thrown, what happens when it is caught, and the meaning of the objects that communicate what went wrong.

18.1.1 Throwing an Exception

In C++, an exception is **raised** by **throwing** an expression. The type of the thrown expression, together with the current call chain, determines which **handler** will deal with the exception. The selected handler is the one nearest in the call chain that matches the type of the thrown object. The type and contents of that object allow the throwing part of the program to inform the handling part about what went wrong.

When a `throw` is executed, the statement(s) following the `throw` are not executed. Instead, control is transferred from the `throw` to the matching `catch`. That `catch` might be local to the same function or might be in a function that directly or indirectly called the function in which the exception occurred. The fact that control passes from one location to another has two important implications:

- Functions along the call chain may be prematurely exited.
- When a handler is entered, objects created along the call chain will have been destroyed.

Because the statements following a `throw` are not executed, a `throw` is like a `return`: It is usually part of a conditional statement or is the last (or only) statement in a function.

Stack Unwinding

When an exception is thrown, execution of the current function is suspended and the search for a matching `catch` clause begins. If the `throw` appears inside a **try block**, the `catch` clauses associated with that `try` are examined. If a matching `catch` is found, the exception is handled by that `catch`. Otherwise, if the `try` was itself nested inside another `try`, the search continues through the `catch` clauses of the enclosing `trys`. If no matching `catch` is found, the current function is exited, and the search continues in the calling function.

If the call to the function that threw is in a `try` block, then the `catch` clauses associated with that `try` are examined. If a matching `catch` is found, the exception is handled. Otherwise, if that `try` was nested, the `catch` clauses of the enclosing `trys` are searched. If no `catch` is found, the calling function is also exited. The search continues in the function that called the just exited one, and so on.

This process, known as **stack unwinding**, continues up the chain of nested function calls until a `catch` clause for the exception is found, or the main function itself is exited without having found a matching `catch`.

Assuming a matching `catch` is found, that `catch` is entered, and the program continues by executing the code inside that `catch`. When the `catch` completes, execution continues at the point immediately after the last `catch` clause associated with that `try` block.

If no matching `catch` is found, the program is exited. Exceptions are intended for events that prevent the program from continuing normally. Therefore, once an exception is raised, it cannot remain unhandled. If no matching `catch` is found, the program calls the library **terminate** function. As its name implies, `terminate` stops execution of the program.



An exception that is not caught terminates the program.

Objects Are Automatically Destroyed during Stack Unwinding

During stack unwinding, blocks in the call chain may be exited prematurely. In general, these blocks will have created local objects. Ordinarily, local objects are destroyed when the block in which they are created is exited. Stack unwinding is no exception. When a block is exited during stack unwinding, the compiler guarantees that objects created in that block are properly destroyed. If a local object is of class type, the destructor for that object is called automatically. As usual, the compiler does no work to destroy objects of built-in type.

If an exception occurs in a constructor, then the object under construction might be only partially constructed. Some of its members might have been initialized, but others might not have been initialized before the exception occurred. Even if the object is only partially constructed, we are guaranteed that the constructed members will be properly destroyed.

Similarly, an exception might occur during initialization of the elements of an array or a library container type. Again, we are guaranteed that the elements (if any) that were constructed before the exception occurred will be destroyed.

Destructors and Exceptions

The fact that destructors are run—but code inside a function that frees a resource may be bypassed—affects how we structure our programs. As we saw in § 12.1.4 (p. 467), if a block allocates a resource, and an exception occurs before the code that frees that resource, the code to free the resource will not be executed. On the other hand, resources allocated by an object of class type generally will be freed by their destructor. By using classes to control resource allocation, we ensure that resources are properly freed, whether a function ends normally or via an exception.

The fact that destructors are run during stack unwinding affects how we write destructors. During stack unwinding, an exception has been raised but is not yet handled. If a new exception is thrown during stack unwinding and not caught in the function that threw it, `terminate` is called. Because destructors may be invoked during stack unwinding, they should never throw exceptions that the destructor itself does not handle. That is, if a destructor does an operation that might throw, it should wrap that operation in a `try` block and handle it locally to the destructor.

In practice, because destructors free resources, it is unlikely that they will throw exceptions. All of the standard library types guarantee that their destructors will not raise an exception.



During stack unwinding, destructors are run on local objects of class type. Because destructors are run automatically, they should not throw. If, during stack unwinding, a destructor throws an exception that it does not also catch, the program will be terminated.

The Exception Object

The compiler uses the thrown expression to copy initialize (§ 13.1.1, p. 497) a special object known as the **exception object**. As a result, the expression in a `throw` must have a complete type (§ 7.3.3, p. 278). Moreover, if the expression has class type, that class must have an accessible destructor and an accessible copy or move constructor. If the expression has an array or function type, the expression is converted to its corresponding pointer type.

The exception object resides in space, managed by the compiler, that is guaranteed to be accessible to whatever `catch` is invoked. The exception object is destroyed after the exception is completely handled.

As we've seen, when an exception is thrown, blocks along the call chain are exited until a matching handler is found. When a block is exited, the memory used by the local objects in that block is freed. As a result, it is almost certainly an error to throw a pointer to a local object. It is an error for the same reasons that it is an error to return a pointer to a local object (§ 6.3.2, p. 225) from a function. If the pointer points to an object in a block that is exited before the `catch`, then that local object will have been destroyed before the `catch`.

When we throw an expression, the static, compile-time type (§ 15.2.3, p. 601) of that expression determines the type of the exception object. This point is essential to keep in mind, because many applications throw expressions whose type comes

from an inheritance hierarchy. If a `throw` expression dereferences a pointer to a base-class type, and that pointer points to a derived-type object, then the thrown object is sliced down (§ 15.2.3, p. 603); only the base-class part is thrown.



Throwing a pointer requires that the object to which the pointer points exist wherever the corresponding handler resides.

EXERCISES SECTION 18.1.1

Exercise 18.1: What is the type of the exception object in the following throws?

(a) `range_error r("error");` (b) `exception *p = &r;`
 `throw r;` `throw *p;`

What would happen if the `throw` in (b) were written as `throw p`?

Exercise 18.2: Explain what happens if an exception occurs at the indicated point:

```
void exercise(int *b, int *e)
{
    vector<int> v(b, e);
    int *p = new int[v.size()];
    ifstream in("ints");
    // exception occurs here
}
```

Exercise 18.3: There are two ways to make the previous code work correctly if an exception is thrown. Describe them and implement them.

18.1.2 Catching an Exception

The **exception declaration** in a **catch clause** looks like a function parameter list with exactly one parameter. As in a parameter list, we can omit the name of the catch parameter if the catch has no need to access the thrown expression.

The type of the declaration determines what kinds of exceptions the handler can catch. The type must be a complete type (§ 7.3.3, p. 278). The type can be an lvalue reference but may not be an rvalue reference (§ 13.6.1, p. 532).

When a `catch` is entered, the parameter in its exception declaration is initialized by the exception object. As with function parameters, if the `catch` parameter has a nonreference type, then the parameter in the `catch` is a copy of the exception object; changes made to the parameter inside the `catch` are made to a local copy, not to the exception object itself. If the parameter has a reference type, then like any reference parameter, the `catch` parameter is just another name for the exception object. Changes made to the parameter are made to the exception object.

Also like a function parameter, a `catch` parameter that has a base-class type can be initialized by an exception object that has a type derived from the parameter type. If the `catch` parameter has a nonreference type, then the exception object

will be sliced down (§ 15.2.3, p. 603), just as it would be if such an object were passed to an ordinary function by value. On the other hand, if the parameter is a reference to a base-class type, then the parameter is bound to the exception object in the usual way.

Again, as with a function parameter, the static type of the exception declaration determines the actions that the `catch` may perform. If the `catch` parameter has a base-class type, then the `catch` cannot use any members that are unique to the derived type.



Ordinarily, a `catch` that takes an exception of a type related by inheritance ought to define its parameter as a reference.

Finding a Matching Handler

During the search for a matching `catch`, the `catch` that is found is not necessarily the one that matches the exception best. Instead, the selected `catch` is the *first* one that matches the exception at all. As a consequence, in a list of `catch` clauses, the most specialized `catch` must appear first.

Because `catch` clauses are matched in the order in which they appear, programs that use exceptions from an inheritance hierarchy must order their `catch` clauses so that handlers for a derived type occur before a `catch` for its base type.

The rules for when an exception matches a `catch` exception declaration are much more restrictive than the rules used for matching arguments with parameter types. Most conversions are not allowed—the types of the exception and the `catch` declaration must match exactly with only a few possible differences:

- Conversions from `nonconst` to `const` are allowed. That is, a `throw` of a `nonconst` object can match a `catch` specified to take a reference to `const`.
- Conversions from derived type to base type are allowed.
- An array is converted to a pointer to the type of the array; a function is converted to the appropriate pointer to function type.

No other conversions are allowed to match a `catch`. In particular, neither the standard arithmetic conversions nor conversions defined for class types are permitted.



Multiple `catch` clauses with types related by inheritance must be ordered from most derived type to least derived.

Rethrow

Sometimes a single `catch` cannot completely handle an exception. After some corrective actions, a `catch` may decide that the exception must be handled by a function further up the call chain. A `catch` passes its exception out to another `catch` by **rethrowing** the exception. A rethrow is a `throw` that is not followed by an expression:

```
throw;
```

An empty `throw` can appear only in a `catch` or in a function called (directly or indirectly) from a `catch`. If an empty `throw` is encountered when a handler is not active, `terminate` is called.

A `rethrow` does not specify an expression; the (current) exception object is passed up the chain.

In general, a `catch` might change the contents of its parameter. If, after changing its parameter, the `catch` rethrows the exception, then those changes will be propagated only if the `catch`'s exception declaration is a reference:

```
catch (my_error &eObj) {           // specifier is a reference type
    eObj.status = errCodes::severeErr; // modifies the exception object
    throw; // the status member of the exception object is severeErr
} catch (other_error eObj) { // specifier is a nonreference type
    eObj.status = errCodes::badErr; // modifies the local copy only
    throw; // the status member of the exception object is unchanged
}
```

The Catch-All Handler

Sometimes we want to catch any exception that might occur, regardless of type. Catching every possible exception can be a problem: Sometimes we don't know what types might be thrown. Even when we do know all the types, it may be tedious to provide a specific `catch` clause for every possible exception. To catch all exceptions, we use an ellipsis for the exception declaration. Such handlers, sometimes known as **catch-all** handlers, have the form `catch (...)`. A catch-all clause matches any type of exception.

A `catch (...)` is often used in combination with a `rethrow` expression. The `catch` does whatever local work can be done and then rethrows the exception:

```
void manip() {
    try {
        // actions that cause an exception to be thrown
    }
    catch (...) {
        // work to partially handle the exception
        throw;
    }
}
```

A `catch (...)` clause can be used by itself or as one of several `catch` clauses.



If a `catch (...)` is used in combination with other `catch` clauses, it must be last. Any `catch` that follows a catch-all can never be matched.

18.1.3 Function `try` Blocks and Constructors

In general, exceptions can occur at any point in the program's execution. In particular, an exception might occur while processing a constructor initializer. Constructor initializers execute before the constructor body is entered. A `catch` inside

EXERCISES SECTION 18.1.2

Exercise 18.4: Looking ahead to the inheritance hierarchy in Figure 18.1 (p. 783), explain what's wrong with the following `try` block. Correct it.

```
try {
    // use of the C++ standard library
} catch(exception) {
    // ...
} catch(const runtime_error &re) {
    // ...
} catch(overflow_error eobj) { /* ... */ }
```

Exercise 18.5: Modify the following `main` function to catch any of the exception types shown in Figure 18.1 (p. 783):

```
int main() {
    // use of the C++ standard library
}
```

The handlers should print the error message associated with the exception before calling `abort` (defined in the header `cstdlib`) to terminate `main`.

Exercise 18.6: Given the following exception types and `catch` clauses, write a `throw` expression that creates an exception object that can be caught by each `catch` clause:

```
(a) class exceptionType { };
    catch(exceptionType *pet) { }
(b) catch(...) { }
(c) typedef int EXCPTYPE;
    catch(EXCPTYPE) { }
```

the constructor body can't handle an exception thrown by a constructor initializer because a `try` block inside the constructor body would not yet be in effect when the exception is thrown.

To handle an exception from a constructor initializer, we must write the constructor as a **function try block**. A function try block lets us associate a group of `catch` clauses with the initialization phase of a constructor (or the destruction phase of a destructor) as well as with the constructor's (or destructor's) function body. As an example, we might wrap the `Blob` constructors (§ 16.1.2, p. 662) in a function try block:

```
template <typename T>
Blob<T>::Blob(std::initializer_list<T> il) try :
    data(std::make_shared<std::vector<T>>(il)) {
    /* empty body */
} catch(const std::bad_alloc &e) { handle_out_of_memory(e); }
```

Notice that the keyword `try` appears before the colon that begins the constructor initializer list and before the curly brace that forms the (in this case empty) constructor function body. The `catch` associated with this `try` can be used to handle

exceptions thrown either from within the member initialization list or from within the constructor body.

It is worth noting that an exception can happen while initializing the constructor's parameters. Such exceptions are *not* part of the function `try` block. The function `try` block handles only exceptions that occur once the constructor begins executing. As with any other function call, if an exception occurs during parameter initialization, that exception is part of the calling expression and is handled in the caller's context.



The only way for a constructor to handle an exception from a constructor initializer is to write the constructor as a function `try` block.

EXERCISES SECTION 18.1.3

Exercise 18.7: Define your `Blob` and `BlobPtr` classes from Chapter 16 to use function `try` blocks for their constructors.

18.1.4 The `noexcept` Exception Specification

It can be helpful both to users and to the compiler to know that a function will not throw any exceptions. Knowing that a function will not throw simplifies the task of writing code that calls that function. Moreover, if the compiler knows that no exceptions will be thrown, it can (sometimes) perform optimizations that must be suppressed if code might throw.

Under the new standard, a function can specify that it does not throw exceptions by providing a **`noexcept` specification**. The keyword `noexcept` following the function parameter list indicates that the function won't throw:

C++
11

```
void recoup(int) noexcept;    // won't throw
void alloc(int);              // might throw
```

These declarations say that `recoup` will not throw any exceptions and that `alloc` might. We say that `recoup` has a **nonthrowing specification**.

The `noexcept` specifier must appear on all of the declarations and the corresponding definition of a function or on none of them. The specifier precedes a trailing return (§ 6.3.3, p. 229). We may also specify `noexcept` on the declaration and definition of a function pointer. It may not appear in a `typedef` or type alias. In a member function the `noexcept` specifier follows any `const` or reference qualifiers, and it precedes `final`, `override`, or `= 0` on a virtual function.

Violating the Exception Specification

It is important to understand that the compiler does not check the `noexcept` specification at compile time. In fact, the compiler is not permitted to reject a function with a `noexcept` specifier merely because it contains a `throw` or calls a function that might throw (however, kind compilers will warn about such usages):

```
// this function will compile, even though it clearly violates its exception specification
void f() noexcept           // promises not to throw any exception
{
    throw exception();      // violates the exception specification
}
```

As a result, it is possible that a function that claims it will not throw will in fact throw. If a `noexcept` function does throw, `terminate` is called, thereby enforcing the promise not to throw at run time. It is unspecified whether the stack is unwound. As a result, `noexcept` should be used in two cases: if we are confident that the function won't throw, and/or if we don't know what we'd do to handle the error anyway.

Specifying that a function won't throw effectively promises the *callers* of the nonthrowing function that they will never need to deal with exceptions. Either the function won't throw, or the whole program will terminate; the caller escapes responsibility either way.



WARNING

The compiler in general cannot, and does not, verify exception specifications at compile time.

BACKWARD COMPATIBILITY: EXCEPTION SPECIFICATIONS

Earlier versions of C++ had a more elaborate scheme of exception specifications that allowed us to specify the types of exceptions that a function might throw. A function can specify the keyword `throw` followed by a parenthesized list of types that the function might throw. The `throw` specifier appeared in the same place as the `noexcept` specifier does in the current language.

This approach was never widely used and has been deprecated in the current standard. Although these more elaborate specifiers have been deprecated, there is one use of the old scheme that is in widespread use. A function that is designated by `throw()` promises not to throw any exceptions:

```
void recoup(int) noexcept; // recoup doesn't throw
void recoup(int) throw();  // equivalent declaration
```

These declarations of `recoup` both say that `recoup` won't throw.

Arguments to the `noexcept` Specification

The `noexcept` specifier takes an optional argument that must be convertible to `bool`: If the argument is `true`, then the function won't throw; if the argument is `false`, then the function might throw:

```
void recoup(int) noexcept(true); // recoup won't throw
void alloc(int) noexcept(false); // alloc can throw
```

The `noexcept` Operator

C++
11

Arguments to the `noexcept` specifier are often composed using the **`noexcept` operator**. The `noexcept` operator is a unary operator that returns a `bool` rvalue

constant expression that indicates whether a given expression might throw. Like `sizeof` (§ 4.9, p. 156), `noexcept` does not evaluate its operand.

For example, this expression yields `true`:

```
noexcept(recoup(i)) // true if calling recoup can't throw, false otherwise
```

because we declared `recoup` with a `noexcept` specifier. More generally,

```
noexcept(e)
```

is `true` if all the functions called by `e` have nonthrowing specifications and `e` itself does not contain a `throw`. Otherwise, `noexcept(e)` returns `false`.

We can use the `noexcept` operator to form an exception specifier as follows:

```
void f() noexcept(noexcept(g())); // f has same exception specifier as g
```

If the function `g` promises not to throw, then `f` also is nonthrowing. If `g` has no exception specifier, or has an exception specifier that allows exceptions, then `f` also might throw.



`noexcept` has two meanings: It is an exception specifier when it follows a function's parameter list, and it is an operator that is often used as the `bool` argument to a `noexcept` exception specifier.

Exception Specifications and Pointers, Virtuals, and Copy Control

Although the `noexcept` specifier is not part of a function's type, whether a function has an exception specification affects the use of that function.

A pointer to function and the function to which that pointer points must have compatible specifications. That is, if we declare a pointer that has a nonthrowing exception specification, we can use that pointer only to point to similarly qualified functions. A pointer that specifies (explicitly or implicitly) that it might throw can point to any function, even if that function includes a promise not to throw:

```
// both recoup and pf1 promise not to throw
void (*pf1)(int) noexcept = recoup;
// ok: recoup won't throw; it doesn't matter that pf2 might
void (*pf2)(int) = recoup;
pf1 = alloc; // error: alloc might throw but pf1 said it wouldn't
pf2 = alloc; // ok: both pf2 and alloc might throw
```

If a virtual function includes a promise not to throw, the inherited virtuals must also promise not to throw. On the other hand, if the base allows exceptions, it is okay for the derived functions to be more restrictive and promise not to throw:

```
class Base {
public:
    virtual double f1(double) noexcept; // doesn't throw
    virtual int f2() noexcept(false);   // can throw
    virtual void f3();                  // can throw
};
```

```

class Derived : public Base {
public:
    double f1(double);           // error: Base::f1 promises not to throw
    int f2() noexcept(false);    // ok: same specification as Base::f2
    void f3() noexcept;          // ok: Derived f3 is more restrictive
};

```

When the compiler synthesizes the copy-control members, it generates an exception specification for the synthesized member. If all the corresponding operation for all the members and base classes promise not to throw, then the synthesized member is `noexcept`. If any function invoked by the synthesized member can throw, then the synthesized member is `noexcept(false)`. Moreover, if we do not provide an exception specification for a destructor that we do define, the compiler synthesizes one for us. The compiler generates the same specification as it would have generated had it synthesized the destructor for that class.

EXERCISES SECTION 18.1.4

Exercise 18.8: Review the classes you’ve written and add appropriate exception specifications to their constructors and destructors. If you think one of your destructors might throw, change the code so that it cannot throw.

18.1.5 Exception Class Hierarchies

The standard-library exception classes (§ 5.6.3, p. 197) form the inheritance hierarchy (Chapter 15) as shown in Figure 18.1.

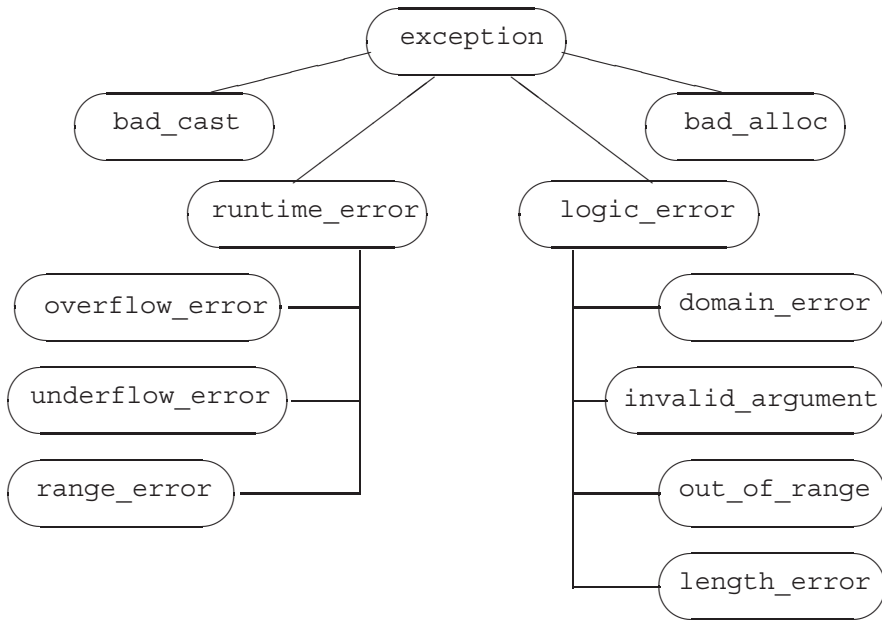
The only operations that the exception types define are the copy constructor, copy-assignment operator, a virtual destructor, and a virtual member named `what`. The `what` function returns a `const char*` that points to a null-terminated character array, and is guaranteed not to throw any exceptions.

The `exception`, `bad_cast`, and `bad_alloc` classes also define a default constructor. The `runtime_error` and `logic_error` classes do not have a default constructor but do have constructors that take a C-style character string or a library string argument. Those arguments are intended to give additional information about the error. In these classes, `what` returns the message used to initialize the exception object. Because `what` is virtual, if we catch a reference to the base-type, a call to the `what` function will execute the version appropriate to the dynamic type of the exception object.

Exception Classes for a Bookstore Application

Applications often extend the exception hierarchy by defining classes derived from `exception` (or from one of the library classes derived from `exception`). These application-specific classes represent exceptional conditions specific to the application domain.

Figure 18.1: Standard exception Class Hierarchy



If we were building a real bookstore application, our classes would have been much more complicated than the ones presented in this Primer. One such complexity would be how these classes handled exceptions. In fact, we probably would have defined our own hierarchy of exceptions to represent application-specific problems. Our design might include classes such as

```

// hypothetical exception classes for a bookstore application
class out_of_stock: public std::runtime_error {
public:
    explicit out_of_stock(const std::string &s):
        std::runtime_error(s) { }
};

class isbn_mismatch: public std::logic_error {
public:
    explicit isbn_mismatch(const std::string &s):
        std::logic_error(s) { }
    isbn_mismatch(const std::string &s,
        const std::string &lhs, const std::string &rhs):
        std::logic_error(s, left(lhs), right(rhs)) { }
    const std::string left, right;
};
  
```

Our application-specific exception types inherit them from the standard exception classes. As with any hierarchy, we can think of the exception classes as being

organized into layers. As the hierarchy becomes deeper, each layer becomes a more specific exception. For example, the first and most general layer of the hierarchy is represented by class `exception`. All we know when we catch an object of type `exception` is that something has gone wrong.

The second layer specializes `exception` into two broad categories: run-time or logic errors. Run-time errors represent things that can be detected only when the program is executing. Logic errors are, in principle, errors that we could have detected in our application.

Our bookstore exception classes further refine these categories. The class named `out_of_stock` represents something, particular to our application, that can go wrong at run time. It would be used to signal that an order cannot be fulfilled. The class `isbn_mismatch` represents a more particular form of `logic_error`. In principle, a program could prevent and handle this error by comparing the results of `isbn()` on the objects.

Using Our Own Exception Types

We use our own exception classes in the same way that we use one of the standard library classes. One part of the program throws an object of one of these types, and another part catches and handles the indicated problem. As an example, we might define the compound addition operator for our `Sales_data` class to throw an error of type `isbn_mismatch` if it detected that the ISBNs didn't match:

```
// throws an exception if both objects do not refer to the same book
Sales_data&
Sales_data::operator+=(const Sales_data& rhs)
{
    if (isbn() != rhs.isbn())
        throw isbn_mismatch("wrong isbn", isbn(), rhs.isbn());
    units_sold += rhs.units_sold;
    revenue += rhs.revenue;
    return *this;
}
```

Code that uses the compound addition operator (or ordinary addition operator, which itself uses the compound addition operator) can detect this error, write an appropriate error message, and continue:

```
// use the hypothetical bookstore exceptions
Sales_data item1, item2, sum;
while (cin >> item1 >> item2) { // read two transactions
    try {
        sum = item1 + item2; // calculate their sum
        // use sum
    } catch (const isbn_mismatch &e) {
        cerr << e.what() << ": left isbn(" << e.left
            << ") right isbn(" << e.right << ")" << endl;
    }
}
```

EXERCISES SECTION 18.1.5

Exercise 18.9: Define the bookstore exception classes described in this section and rewrite your `Sales_data` compound assignment operator to throw an exception.

Exercise 18.10: Write a program that uses the `Sales_data` addition operator on objects that have differing ISBNs. Write two versions of the program: one that handles the exception and one that does not. Compare the behavior of the programs so that you become familiar with what happens when an uncaught exception occurs.

Exercise 18.11: Why is it important that the `what` function doesn't throw?

18.2 Namespaces

Large programs tend to use independently developed libraries. Such libraries also tend to define a large number of global names, such as classes, functions, and templates. When an application uses libraries from many different vendors, it is almost inevitable that some of these names will clash. Libraries that put names into the global namespace are said to cause **namespace pollution**.

Traditionally, programmers avoided namespace pollution by using very long names for the global entities they defined. Those names often contained a prefix indicating which library defined the name:

```
class cplusplus_primer_Query { ... };
string cplusplus_primer_make_plural(size_t, string&);
```

This solution is far from ideal: It can be cumbersome for programmers to write and read programs that use such long names.

Namespaces provide a much more controlled mechanism for preventing name collisions. Namespaces partition the global namespace. A namespace is a scope. By defining a library's names inside a namespace, library authors (and users) can avoid the limitations inherent in global names.

18.2.1 Namespace Definitions

A namespace definition begins with the keyword `namespace` followed by the namespace name. Following the namespace name is a sequence of declarations and definitions delimited by curly braces. Any declaration that can appear at global scope can be put into a namespace: classes, variables (with their initializations), functions (with their definitions), templates, and other namespaces:

```
namespace cplusplus_primer {
    class Sales_data { /* ... */};
    Sales_data operator+(const Sales_data&,
                        const Sales_data&);

    class Query { /* ... */};
    class Query_base { /* ... */};
} // like blocks, namespaces do not end with a semicolon
```

This code defines a namespace named `cplusplus_primer` with four members: three classes and an overloaded `+` operator.

As with any name, a namespace name must be unique within the scope in which the namespace is defined. Namespaces may be defined at global scope or inside another namespace. They may not be defined inside a function or a class.



A namespace scope does not end with a semicolon.

Each Namespace Is a Scope

As is the case for any scope, each name in a namespace must refer to a unique entity within that namespace. Because different namespaces introduce different scopes, different namespaces may have members with the same name.

Names defined in a namespace may be accessed directly by other members of the namespace, including scopes nested within those members. Code outside the namespace must indicate the namespace in which the name is defined:

```
cplusplus_primer::Query q =
    cplusplus_primer::Query("hello");
```

If another namespace (say, `AddisonWesley`) also provides a `Query` class and we want to use that class instead of the one defined in `cplusplus_primer`, we can do so by modifying our code as follows:

```
AddisonWesley::Query q = AddisonWesley::Query("hello");
```

Namespaces Can Be Discontiguous

As we saw in § 16.5 (p. 709), unlike other scopes, a namespace can be defined in several parts. Writing a namespace definition:

```
namespace nsp {
    // declarations
}
```

either defines a new namespace named `nsp` or adds to an existing one. If the name `nsp` does not refer to a previously defined namespace, then a new namespace with that name is created. Otherwise, this definition opens an existing namespace and adds declarations to that already existing namespace.

The fact that namespace definitions can be discontiguous lets us compose a namespace from separate interface and implementation files. Thus, a namespace can be organized in the same way that we manage our own class and function definitions:

- Namespace members that define classes, and declarations for the functions and objects that are part of the class interface, can be put into header files. These headers can be included by files that use those namespace members.
- The definitions of namespace members can be put in separate source files.

Organizing our namespaces this way also satisfies the requirement that various entities—non-inline functions, static data members, variables, and so forth—may be defined only once in a program. This requirement applies equally to names defined in a namespace. By separating the interface and implementation, we can ensure that the functions and other names we need are defined only once, but the same declaration will be seen whenever the entity is used.



Namespaces that define multiple, unrelated types should use separate files to represent each type (or each collection of related types) that the namespace defines.

Defining the Primer Namespace

Using this strategy for separating interface and implementation, we might define the `cplusplus_primer` library in several separate files. The declarations for `Sales_data` and its related functions would be placed in `Sales_data.h`, those for the `Query` classes of Chapter 15 in `Query.h`, and so on. The corresponding implementation files would be in files such as `Sales_data.cc` and `Query.cc`:

```
// --- Sales_data.h ---
// #includes should appear before opening the namespace
#include <string>
namespace cplusplus_primer {
    class Sales_data { /* ... */};
    Sales_data operator+(const Sales_data&,
                        const Sales_data&);
    // declarations for the remaining functions in the Sales_data interface
}
// --- Sales_data.cc ---
// be sure any #includes appear before opening the namespace
#include "Sales_data.h"
namespace cplusplus_primer {
    // definitions for Sales_data members and overloaded operators
}
```

A program using our library would include whichever headers it needed. The names in those headers are defined inside the `cplusplus_primer` namespace:

```
// --- user.cc ---
// names in the Sales_data.h header are in the cplusplus_primer namespace
#include "Sales_data.h"
int main()
{
    using cplusplus_primer::Sales_data;
    Sales_data trans1, trans2;
    // ...
    return 0;
}
```

This program organization gives the developers and the users of our library the needed modularity. Each class is still organized into its own interface and

implementation files. A user of one class need not compile names related to the others. We can hide the implementations from our users, while allowing the files `Sales_data.cc` and `user.cc` to be compiled and linked into one program without causing any compile-time or link-time errors. Developers of the library can work independently on the implementation of each type.

It is worth noting that ordinarily, we do not put a `#include` inside the namespace. If we did, we would be attempting to define all the names in that header as members of the enclosing namespace. For example, if our `Sales_data.h` file opened the `cplusplus_primer` before including the `string` header our program would be in error. It would be attempting to define the `std` namespace nested inside `cplusplus_primer`.

Defining Namespace Members

Assuming the appropriate declarations are in scope, code inside a namespace may use the short form for names defined in the same (or in an enclosing) namespace:

```
#include "Sales_data.h"
namespace cplusplus_primer {    // reopen cplusplus_primer
    // members defined inside the namespace may use unqualified names
    std::istream&
    operator>>(std::istream& in, Sales_data& s) { /* ... */ }
}
```

It is also possible to define a namespace member outside its namespace definition. The namespace declaration of the name must be in scope, and the definition must specify the namespace to which the name belongs:

```
// namespace members defined outside the namespace must use qualified names
cplusplus_primer::Sales_data
cplusplus_primer::operator+(const Sales_data& lhs,
                           const Sales_data& rhs)
{
    Sales_data ret(lhs);
    // ...
}
```

As with class members defined outside a class, once the fully qualified name is seen, we are in the scope of the namespace. Inside the `cplusplus_primer` namespace, we can use other namespace member names without qualification. Thus, even though `Sales_data` is a member of the `cplusplus_primer` namespace, we can use its unqualified name to define the parameters in this function.

Although a namespace member can be defined outside its namespace, such definitions must appear in an enclosing namespace. That is, we can define the `Sales_data` `operator+` inside the `cplusplus_primer` namespace or at global scope. We cannot define this operator in an unrelated namespace.

Template Specializations

Template specializations must be defined in the same namespace that contains the original template (§ 16.5, p. 709). As with any other namespace name, so long as

we have declared the specialization inside the namespace, we can define it outside the namespace:

```
// we must declare the specialization as a member of std
namespace std {
    template <> struct hash<Sales_data>;
}
// having added the declaration for the specialization to std
// we can define the specialization outside the std namespace
template <> struct std::hash<Sales_data>
{
    size_t operator()(const Sales_data& s) const
    { return hash<string>()(s.bookNo) ^
        hash<unsigned>()(s.units_sold) ^
        hash<double>()(s.revenue); }
    // other members as before
};
```

The Global Namespace

Names defined at global scope (i.e., names declared outside any class, function, or namespace) are defined inside the **global namespace**. The global namespace is implicitly declared and exists in every program. Each file that defines entities at global scope (implicitly) adds those names to the global namespace.

The scope operator can be used to refer to members of the global namespace. Because the global namespace is implicit, it does not have a name; the notation

```
::member_name
```

refers to a member of the global namespace.

Nested Namespaces

A nested namespace is a namespace defined inside another namespace:

```
namespace cplusplus_primer {
    // first nested namespace: defines the Query portion of the library
    namespace QueryLib {
        class Query { /* ... */ };
        Query operator&(const Query&, const Query&);
        // ...
    }
    // second nested namespace: defines the Sales_data portion of the library
    namespace Bookstore {
        class Quote { /* ... */ };
        class Disc_quote : public Quote { /* ... */ };
        // ...
    }
}
```

The `cplusplus_primer` namespace now contains two nested namespaces: the namespaces named `QueryLib` and `Bookstore`.

A nested namespace is a nested scope—its scope is nested within the namespace that contains it. Nested namespace names follow the normal rules: Names declared in an inner namespace hide declarations of the same name in an outer namespace. Names defined inside a nested namespace are local to that inner namespace. Code in the outer parts of the enclosing namespace may refer to a name in a nested namespace only through its qualified name: For example, the name of the class declared in the nested namespace `QueryLib` is

```
cplusplus_primer::QueryLib::Query
```

Inline Namespaces

**C++
11**

The new standard introduced a new kind of nested namespace, an **inline namespace**. Unlike ordinary nested namespaces, names in an inline namespace can be used as if they were direct members of the enclosing namespace. That is, we need not qualify names from an inline namespace by their namespace name. We can access them using only the name of the enclosing namespace.

An inline namespace is defined by preceding the keyword `namespace` with the keyword `inline`:

```
inline namespace FifthEd {
    // namespace for the code from the Primer Fifth Edition
}
namespace FifthEd { // implicitly inline
    class Query_base { /* ... */};
    // other Query-related declarations
}
```

The keyword must appear on the first definition of the namespace. If the namespace is later reopened, the keyword `inline` need not be, but may be, repeated.

Inline namespaces are often used when code changes from one release of an application to the next. For example, we can put all the code from the current edition of the Primer into an inline namespace. Code for previous versions would be in non-inlined namespaces:

```
namespace FourthEd {
    class Item_base { /* ... */};
    class Query_base { /* ... */};
    // other code from the Fourth Edition
}
```

The overall `cplusplus_primer` namespace would include the definitions of both namespaces. For example, assuming that each namespace was defined in a header with the corresponding name, we'd define `cplusplus_primer` as follows:

```
namespace cplusplus_primer {
    #include "FifthEd.h"
    #include "FourthEd.h"
}
```


Because `FifthEd` is inline, code that refers to `cplusplus_primer::` will get the version from that namespace. If we want the earlier edition code, we can access it as we would any other nested namespace, by using the names of all the enclosing namespaces: for example, `cplusplus_primer::FourthEd::Query_base`.

Unnamed Namespaces

An **unnamed namespace** is the keyword `namespace` followed immediately by a block of declarations delimited by curly braces. Variables defined in an unnamed namespace have static lifetime: They are created before their first use and destroyed when the program ends.

An unnamed namespace may be discontinuous within a given file but does not span files. Each file has its own unnamed namespace. If two files contain unnamed namespaces, those namespaces are unrelated. Both unnamed namespaces can define the same name; those definitions would refer to different entities. If a header defines an unnamed namespace, the names in that namespace define different entities local to each file that includes the header.



Unlike other namespaces, an unnamed namespace is local to a particular file and never spans multiple files.

Names defined in an unnamed namespace are used directly; after all, there is no namespace name with which to qualify them. It is not possible to use the scope operator to refer to members of unnamed namespaces.

Names defined in an unnamed namespace are in the same scope as the scope at which the namespace is defined. If an unnamed namespace is defined at the outermost scope in the file, then names in the unnamed namespace must differ from names defined at global scope:

```
int i;    // global declaration for i
namespace {
    int i;
}
// ambiguous: defined globally and in an unnested, unnamed namespace
i = 10;
```

In all other ways, the members of an unnamed namespace are normal program entities. An unnamed namespace, like any other namespace, may be nested inside another namespace. If the unnamed namespace is nested, then names in it are accessed in the normal way, using the enclosing namespace name(s):

```
namespace local {
    namespace {
        int i;
    }
}
// ok: i defined in a nested unnamed namespace is distinct from global i
local::i = 42;
```

UNNAMED NAMESPACES REPLACE FILE STATICS

Prior to the introduction of namespaces, programs declared names as `static` to make them local to a file. The use of *file statics* is inherited from C. In C, a global entity declared `static` is invisible outside the file in which it is declared.



The use of file `static` declarations is deprecated by the C++ standard. File statics should be avoided and unnamed namespaces used instead.

EXERCISES SECTION 18.2.1

Exercise 18.12: Organize the programs you have written to answer the questions in each chapter into their own namespaces. That is, namespace `chapter15` would contain code for the Query programs and `chapter10` would contain the TextQuery code. Using this structure, compile the Query code examples.

Exercise 18.13: When might you use an unnamed namespace?

Exercise 18.14: Suppose we have the following declaration of the operator* that is a member of the nested namespace `mathLib::MatrixLib`:

```
namespace mathLib {
    namespace MatrixLib {
        class matrix { /* ... */ };
        matrix operator*
            (const matrix &, const matrix &);
        // ...
    }
}
```

How would you declare this operator in global scope?

18.2.2 Using Namespace Members

Referring to namespace members as `namespace_name::member_name` is admittedly cumbersome, especially if the namespace name is long. Fortunately, there are ways to make it easier to use namespace members. Our programs have used one of these ways, using declarations (§ 3.1, p. 82). The others, namespace aliases and using directives, will be described in this section.

Namespace Aliases

A **namespace alias** can be used to associate a shorter synonym with a namespace name. For example, a long namespace name such as

```
namespace cplusplus_primer { /* ... */ ;
```

can be associated with a shorter synonym as follows:

```
namespace primer = cplusplus_primer;
```

A namespace alias declaration begins with the keyword `namespace`, followed by the alias name, followed by the `=` sign, followed by the original namespace name and a semicolon. It is an error if the original namespace name has not already been defined as a namespace.

A namespace alias can also refer to a nested namespace:

```
namespace Qlib = cplusplus_primer::QueryLib;  
Qlib::Query q;
```



A namespace can have many synonyms, or aliases. All the aliases and the original namespace name can be used interchangeably.

using Declarations: A Recap

A **using declaration** introduces only one namespace member at a time. It allows us to be very specific regarding which names are used in our programs.

Names introduced in a `using` declaration obey normal scope rules: They are visible from the point of the `using` declaration to the end of the scope in which the declaration appears. Entities with the same name defined in an outer scope are hidden. The unqualified name may be used only within the scope in which it is declared and in scopes nested within that scope. Once the scope ends, the fully qualified name must be used.

A `using` declaration can appear in global, local, namespace, or class scope. In class scope, such declarations may only refer to a base class member (§ 15.5, p. 615).

using Directives

A **using directive**, like a `using` declaration, allows us to use the unqualified form of a namespace name. Unlike a `using` declaration, we retain no control over which names are made visible—they all are.

A `using` directive begins with the keyword `using`, followed by the keyword `namespace`, followed by a namespace name. It is an error if the name is not a previously defined namespace name. A `using` directive may appear in global, local, or namespace scope. It may not appear in a class scope.

These directives make all the names from a specific namespace visible without qualification. The short form names can be used from the point of the `using` directive to the end of the scope in which the `using` directive appears.



WARNING

Providing a `using` directive for namespaces, such as `std`, that our application does not control reintroduces all the name collision problems inherent in using multiple libraries.

using Directives and Scope

The scope of names introduced by a `using` directive is more complicated than the scope of names in `using` declarations. As we've seen, a `using` declaration puts the name in the same scope as that of the `using` declaration itself. It is as if the `using` declaration declares a local alias for the namespace member.

A `using` directive does not declare local aliases. Rather, it has the effect of lifting the namespace members into the nearest scope that contains both the namespace itself and the `using` directive.

This difference in scope between a `using` declaration and a `using` directive stems directly from how these two facilities work. In the case of a `using` declaration, we are simply making name directly accessible in the local scope. In contrast, a `using` directive makes the entire contents of a namespace available. In general, a namespace might include definitions that cannot appear in a local scope. As a consequence, a `using` directive is treated as if it appeared in the nearest enclosing namespace scope.

In the simplest case, assume we have a namespace `A` and a function `f`, both defined at global scope. If `f` has a `using` directive for `A`, then in `f` it will be as if the names in `A` appeared in the global scope prior to the definition of `f`:

```
// namespace A and function f are defined at global scope
namespace A {
    int i, j;
}
void f()
{
    using namespace A;      // injects the names from A into the global scope
    cout << i * j << endl; // uses i and j from namespace A
    // ...
}
```

using Directives Example

Let's look at an example:

```
namespace blip {
    int i = 16, j = 15, k = 23;
    // other declarations
}
int j = 0; // ok: j inside blip is hidden inside a namespace
void manip()
{
    // using directive; the names in blip are "added" to the global scope
    using namespace blip; // clash between ::j and blip::j
                          // detected only if j is used

    ++i; // sets blip::i to 17
    ++j; // error ambiguous: global j or blip::j?
    ++::j; // ok: sets global j to 1
    ++blip::j; // ok: sets blip::j to 16
    int k = 97; // local k hides blip::k
    ++k; // sets local k to 98
}
```

The `using` directive in `manip` makes all the names in `blip` directly accessible; code inside `manip` can refer to the names of these members, using their short form.

The members of `blip` appear as if they were defined in the scope in which both `blip` and `manip` are defined. Assuming `manip` is defined at global scope, then the members of `blip` appear as if they were declared in global scope.

When a namespace is injected into an enclosing scope, it is possible for names in the namespace to conflict with other names defined in that (enclosing) scope. For example, inside `manip`, the `blip` member `j` conflicts with the global object named `j`. Such conflicts are permitted, but to use the name, we must explicitly indicate which version is wanted. Any unqualified use of `j` within `manip` is ambiguous.

To use a name such as `j`, we must use the scope operator to indicate which name is wanted. We would write `::j` to obtain the variable defined in global scope. To use the `j` defined in `blip`, we must use its qualified name, `blip::j`.

Because the names are in different scopes, local declarations within `manip` may hide some of the namespace member names. The local variable `k` hides the namespace member `blip::k`. Referring to `k` within `manip` is not ambiguous; it refers to the local variable `k`.

Headers and `using` Declarations or Directives

A header that has a `using` directive or declaration at its top-level scope injects names into every file that includes the header. Ordinarily, headers should define only the names that are part of its interface, not names used in its own implementation. As a result, header files should not contain `using` directives or `using` declarations except inside functions or namespaces (§ 3.1, p. 83).

CAUTION: AVOID `using` DIRECTIVES

`using` directives, which inject all the names from a namespace, are deceptively simple to use: With only a single statement, all the member names of a namespace are suddenly visible. Although this approach may seem simple, it can introduce its own problems. If an application uses many libraries, and if the names within these libraries are made visible with `using` directives, then we are back to square one, and the global namespace pollution problem reappears.

Moreover, it is possible that a working program will fail to compile when a new version of the library is introduced. This problem can arise if a new version introduces a name that conflicts with a name that the application is using.

Another problem is that ambiguity errors caused by `using` directives are detected only at the point of use. This late detection means that conflicts can arise long after introducing a particular library. If the program begins using a new part of the library, previously undetected collisions may arise.

Rather than relying on a `using` directive, it is better to use a `using` declaration for each namespace name used in the program. Doing so reduces the number of names injected into the namespace. Ambiguity errors caused by `using` declarations are detected at the point of declaration, not use, and so are easier to find and fix.



One place where `using` directives are useful is in the implementation files of the namespace itself.

EXERCISES SECTION 18.2.2

Exercise 18.15: Explain the differences between using declarations and directives.

Exercise 18.16: Explain the following code assuming using declarations for all the members of namespace `Exercise` are located at the location labeled *position 1*. What if they appear at *position 2* instead? Now answer the same question but replace the using declarations with a using directive for namespace `Exercise`.

```
namespace Exercise {
    int ivar = 0;
    double dvar = 0;
    const int limit = 1000;
}
int ivar = 0;
// position 1
void manip() {
    // position 2
    double dvar = 3.1416;
    int iobj = limit + 1;
    ++ivar;
    ++::ivar;
}
```

Exercise 18.17: Write code to test your answers to the previous question.

18.2.3 Classes, Namespaces, and Scope

Name lookup for names used inside a namespace follows the normal lookup rules: The search looks outward through the enclosing scopes. An enclosing scope might be one or more nested namespaces, ending in the all-encompassing global namespace. Only names that have been declared before the point of use that are in blocks that are still open are considered:

```
namespace A {
    int i;
    namespace B {
        int i;           // hides A::i within B
        int j;
        int f1()
        {
            int j;       // j is local to f1 and hides A::B::j
            return i;    // returns B::i
        }
    } // namespace B is closed and names in it are no longer visible
    int f2() {
        return j;        // error: j is not defined
    }
    int j = i;          // initialized from A::i
}
```

When a class is wrapped in a namespace, the normal lookup still happens: When a name is used by a member function, look for that name in the member first, then within the class (including base classes), then look in the enclosing scopes, one or more of which might be a namespace:

```
namespace A {
    int i;
    int k;
    class C1 {
    public:
        C1(): i(0), j(0) { }    // ok: initializes C1::i and C1::j
        int f1() { return k; } // returns A::k
        int f2() { return h; } // error: h is not defined
        int f3();
    private:
        int i;                // hides A::i within C1
        int j;
    };
    int h = i;                // initialized from A::i
}
// member f3 is defined outside class C1 and outside namespace A
int A::C1::f3() { return h; } // ok: returns A::h
```

With the exception of member function definitions that appear inside the class body (§ 7.4.1, p. 283), scopes are always searched upward; names must be declared before they can be used. Hence, the return in `f2` will not compile. It attempts to reference the name `h` from namespace `A`, but `h` has not yet been defined. Had that name been defined in `A` before the definition of `C1`, the use of `h` would be legal. Similarly, the use of `h` inside `f3` is okay, because `f3` is defined after `A::h`.



The order in which scopes are examined to find a name can be inferred from the qualified name of a function. The qualified name indicates, in reverse order, the scopes that are searched.

The qualifiers `A::C1::f3` indicate the reverse order in which the class scopes and namespace scopes are to be searched. The first scope searched is that of the function `f3`. Then the class scope of its enclosing class `C1` is searched. The scope of the namespace `A` is searched last before the scope containing the definition of `f3` is examined.

Argument-Dependent Lookup and Parameters of Class Type



Consider the following simple program:

```
std::string s;
std::cin >> s;
```

As we know, this call is equivalent to (§ 14.1, p. 553):

```
operator>>(std::cin, s);
```

This `operator>>` function is defined by the `string` library, which in turn is defined in the `std` namespace. Yet we can we call `operator>>` without an `std::` qualifier and without a `using` declaration.

We can directly access the output operator because there is an important exception to the rule that names defined in a namespace are hidden. When we pass an object of a class type to a function, the compiler searches the namespace in which the argument's class is defined *in addition* to the normal scope lookup. This exception also applies for calls that pass pointers or references to a class type.

In this example, when the compiler sees the “call” to `operator>>`, it looks for a matching function in the current scope, including the scopes enclosing the output statement. In addition, because the `>>` expression has parameters of class type, the compiler also looks in the namespace(s) in which the types of `cin` and `s` are defined. Thus, for this call, the compiler looks in the `std` namespace, which defines the `istream` and `string` types. When it searches `std`, the compiler finds the `string` output operator function.

This exception in the lookup rules allows nonmember functions that are conceptually part of the interface to a class to be used without requiring a separate `using` declaration. In the absence of this exception to the lookup rules, either we would have to provide an appropriate `using` declaration for the output operator:

```
using std::operator>>;           // needed to allow cin >> s
```

or we would have to use the function-call notation in order to include the namespace qualifier:

```
std::operator>>(std::cin, s);    // ok: explicitly use std::>>
```

There would be no way to use operator syntax. Either of these declarations is awkward and would make simple uses of the IO library more complicated.

Lookup and `std::move` and `std::forward`

Many, perhaps even most, C++ programmers never have to think about argument-dependent lookup. Ordinarily, if an application defines a name that is also defined in the library, one of two things is true: Either normal overloading determines (correctly) whether a particular call is intended for the application version or the one from the library, or the application never intends to use the library function.

Now consider the library `move` and `forward` functions. Both of these functions are template functions, and the library defines versions of them that have a single rvalue reference function parameter. As we've seen, in a function template, an rvalue reference parameter can match any type (§ 16.2.6, p. 690). If our application defines a function named `move` that takes a single parameter, then—no matter what type the parameter has—the application's version of `move` will collide with the library version. Similarly for `forward`.

As a result, name collisions with `move` (and `forward`) are more likely than collisions with other library functions. In addition, because `move` and `forward` do very specialized type manipulations, the chances that an application specifically wants to override the behavior of these functions are pretty small.

The fact that collisions are more likely—and are less likely to be intentional—explains why we suggest always using the fully qualified versions of these names (§ 12.1.5, p. 470). So long as we write `std::move` rather than `move`, we know that we will get the version from the standard library.

Friend Declarations and Argument-Dependent Lookup



Recall that when a class declares a friend, the friend declaration does not make the friend visible (§ 7.2.1, p. 270). However, an otherwise undeclared class or function that is first named in a friend declaration is assumed to be a member of the closest enclosing namespace. The combination of this rule and argument-dependent lookup can lead to surprises:

```
namespace A {
    class C {
        // two friends; neither is declared apart from a friend declaration
        // these functions implicitly are members of namespace A
        friend void f2(); // won't be found, unless otherwise declared
        friend void f(const C&); // found by argument-dependent lookup
    };
}
```

Here, both `f` and `f2` are members of namespace `A`. Through argument-dependent lookup, we can call `f` even if there is no additional declaration for `f`:

```
int main()
{
    A::C cobj;
    f(cobj); // ok: finds A::f through the friend declaration in A::C
    f2(); // error: A::f2 not declared
}
```

Because `f` takes an argument of a class type, and `f` is implicitly declared in the same namespace as `C`, `f` is found when called. Because `f2` has no parameter, it will not be found.

EXERCISES SECTION 18.2.3

Exercise 18.18: Given the following typical definition of `swap` § 13.3 (p. 517), determine which version of `swap` is used if `mem1` is a string. What if `mem1` is an `int`? Explain how name lookup works in both cases.

```
void swap(T v1, T v2)
{
    using std::swap;
    swap(v1.mem1, v2.mem1);
    // swap remaining members of type T
}
```

Exercise 18.19: What if the call to `swap` was `std::swap(v1.mem1, v2.mem1)`?

18.2.4 Overloading and Namespaces

Namespaces have two impacts on function matching (§ 6.4, p. 233). One of these should be obvious: A `using` declaration or directive can add functions to the candidate set. The other is much more subtle.



Argument-Dependent Lookup and Overloading

As we saw in the previous section, name lookup for functions that have class-type arguments includes the namespace in which each argument's class is defined. This rule also impacts how we determine the candidate set. Each namespace that defines a class used as an argument (and those that define its base classes) is searched for candidate functions. Any functions in those namespaces that have the same name as the called function are added to the candidate set. These functions are added *even though they otherwise are not visible at the point of the call*:

```
namespace NS {
    class Quote { /* ... */ };
    void display(const Quote&) { /* ... */ }
}
// Bulk_item's base class is declared in namespace NS
class Bulk_item : public NS::Quote { /* ... */ };
int main() {
    Bulk_item book1;
    display(book1);
    return 0;
}
```

The argument we passed to `display` has class type `Bulk_item`. The candidate functions for the call to `display` are not only the functions with declarations that are in scope where `display` is called, but also the functions in the namespace where `Bulk_item` and its base class, `Quote`, are declared. The function `display(const Quote&)` declared in namespace `NS` is added to the set of candidate functions.

Overloading and `using` Declarations

To understand the interaction between `using` declarations and overloading, it is important to remember that a `using` declaration declares a name, not a specific function (§ 15.6, p. 621):

```
using NS::print(int);    // error: cannot specify a parameter list
using NS::print;         // ok: using declarations specify names only
```

When we write a `using` declaration for a function, all the versions of that function are brought into the current scope.

A `using` declaration incorporates all versions to ensure that the interface of the namespace is not violated. The author of a library provided different functions for a reason. Allowing users to selectively ignore some but not all of the functions from a set of overloaded functions could lead to surprising program behavior.

The functions introduced by a `using` declaration overload any other declarations of the functions with the same name already present in the scope where the `using` declaration appears. If the `using` declaration appears in a local scope, these names hide existing declarations for that name in the outer scope. If the `using` declaration introduces a function in a scope that already has a function of the same name with the same parameter list, then the `using` declaration is in error. Otherwise, the `using` declaration defines additional overloaded instances of the given name. The effect is to increase the set of candidate functions.

Overloading and `using` Directives

A `using` directive lifts the namespace members into the enclosing scope. If a namespace function has the same name as a function declared in the scope at which the namespace is placed, then the namespace member is added to the overload set:

```
namespace libs_R_us {
    extern void print(int);
    extern void print(double);
}
// ordinary declaration
void print(const std::string &);
// this using directive adds names to the candidate set for calls to print:
using namespace libs_R_us;
// the candidates for calls to print at this point in the program are:
//   print(int) from libs_R_us
//   print(double) from libs_R_us
//   print(const std::string &) declared explicitly
void fooBar(int ival)
{
    print("Value: "); // calls global print(const string &)
    print(ival);      // calls libs_R_us::print(int)
}
```

Differently from how `using` declarations work, it is not an error if a `using` directive introduces a function that has the same parameters as an existing function. As with other conflicts generated by `using` directives, there is no problem unless we try to call the function without specifying whether we want the one from the namespace or from the current scope.

Overloading across Multiple `using` Directives

If many `using` directives are present, then the names from each namespace become part of the candidate set:

```
namespace AW {
    int print(int);
}
namespace Primer {
    double print(double);
}
```

```

// using directives create an overload set of functions from different namespaces
using namespace AW;
using namespace Primer;
long double print(long double);
int main() {
    print(1);    // calls AW::print(int)
    print(3.1);  // calls Primer::print(double)
    return 0;
}

```

The overload set for the function `print` in global scope contains the functions `print(int)`, `print(double)`, and `print(long double)`. These functions are all part of the overload set considered for the function calls in `main`, even though these functions were originally declared in different namespace scopes.

EXERCISES SECTION 18.2.4

Exercise 18.20: In the following code, determine which function, if any, matches the call to `compute`. List the candidate and viable functions. What type conversions, if any, are applied to the argument to match the parameter in each viable function?

```

namespace primerLib {
    void compute();
    void compute(const void *);
}
using primerLib::compute;
void compute(int);
void compute(double, double = 3.4);
void compute(char*, char* = 0);
void f()
{
    compute(0);
}

```

What would happen if the `using` declaration were located in `main` before the call to `compute`? Answer the same questions as before.

18.3 Multiple and Virtual Inheritance

Multiple inheritance is the ability to derive a class from more than one direct base class (§ 15.2.2, p. 600). A multiply derived class inherits the properties of all its parents. Although simple in concept, the details of intertwining multiple base classes can present tricky design-level and implementation-level problems.

To explore multiple inheritance, we'll use a pedagogical example of a zoo animal hierarchy. Our zoo animals exist at different levels of abstraction. There are the individual animals, distinguished by their names, such as Ling-ling, Mowgli, and Balou. Each animal belongs to a species; Ling-Ling, for example, is a giant

panda. Species, in turn, are members of families. A giant panda is a member of the bear family. Each family, in turn, is a member of the animal kingdom—in this case, the more limited kingdom of a particular zoo.

We'll define an abstract `ZooAnimal` class to hold information that is common to all the zoo animals and provides the most general interface. The `Bear` class will contain information that is unique to the `Bear` family, and so on.

In addition to the `ZooAnimal` classes, our application will contain auxiliary classes that encapsulate various abstractions such as endangered animals. In our implementation of a `Panda` class, for example, a `Panda` is multiply derived from `Bear` and `Endangered`.

18.3.1 Multiple Inheritance

The derivation list in a derived class can contain more than one base class:

```
class Bear : public ZooAnimal {
class Panda : public Bear, public Endangered { /* ... */ };
```

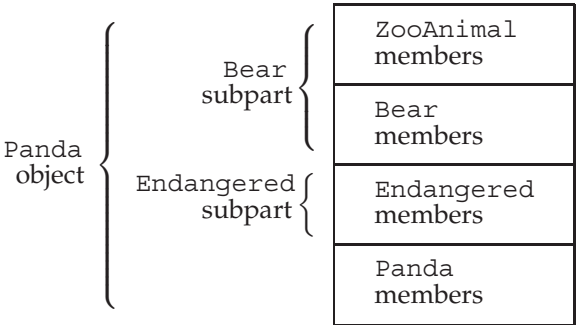
Each base class has an optional access specifier (§ 15.5, p. 612). As usual, if the access specifier is omitted, the specifier defaults to `private` if the `class` keyword is used and to `public` if `struct` is used (§ 15.5, p. 616).

As with single inheritance, the derivation list may include only classes that have been defined and that were not defined as `final` (§ 15.2.2, p. 600). There is no language-imposed limit on the number of base classes from which a class can be derived. A base class may appear only once in a given derivation list.

Multiply Derived Classes Inherit State from Each Base Class

Under multiple inheritance, an object of a derived class contains a subobject for each of its base classes (§ 15.2.2, p. 597). For example, as illustrated in Figure 18.2, a `Panda` object has a `Bear` part (which itself contains a `ZooAnimal` part), an `Endangered` class part, and the nonstatic data members, if any, declared within the `Panda` class.

Figure 18.2: Conceptual Structure of a Panda Object



Derived Constructors Initialize All Base Classes

Constructing an object of derived type constructs and initializes all its base sub-objects. As is the case for inheriting from a single base class (§ 15.2.2, p. 598), a derived type's constructor initializer may initialize only its direct base classes:

```
// explicitly initialize both base classes
Panda::Panda(std::string name, bool onExhibit)
    : Bear(name, onExhibit, "Panda"),
      Endangered(Endangered::critical) { }

// implicitly uses the Bear default constructor to initialize the Bear subobject
Panda::Panda()
    : Endangered(Endangered::critical) { }
```

The constructor initializer list may pass arguments to each of the direct base classes. The order in which base classes are constructed depends on the order in which they appear in the class derivation list. The order in which they appear in the constructor initializer list is irrelevant. A Panda object is initialized as follows:

- ZooAnimal, the ultimate base class up the hierarchy from Panda's first direct base class, Bear, is initialized first.
- Bear, the first direct base class, is initialized next.
- Endangered, the second direct base, is initialized next.
- Panda, the most derived part, is initialized last.

Inherited Constructors and Multiple Inheritance

**C++
11**

Under the new standard, a derived class can inherit its constructors from one or more of its base classes (§ 15.7.4, p. 628). It is an error to inherit the same constructor (i.e., one with the same parameter list) from more than one base class:

```
struct Base1 {
    Base1() = default;
    Base1(const std::string&);
    Base1(std::shared_ptr<int>);
};

struct Base2 {
    Base2() = default;
    Base2(const std::string&);
    Base2(int);
};

// error: D1 attempts to inherit D1::D1(const string&) from both base classes
struct D1: public Base1, public Base2 {
    using Base1::Base1; // inherit constructors from Base1
    using Base2::Base2; // inherit constructors from Base2
};
```

A class that inherits the same constructor from more than one base class must define its own version of that constructor:

```

struct D2: public Base1, public Base2 {
    using Base1::Base1;    // inherit constructors from Base1
    using Base2::Base2;    // inherit constructors from Base2
    // D2 must define its own constructor that takes a string
    D2(const string &s): Base1(s), Base2(s) { }
    D2() = default;        // needed once D2 defines its own constructor
};

```

Destructors and Multiple Inheritance

As usual, the destructor in a derived class is responsible for cleaning up resources allocated by that class only—the members and all the base class(es) of the derived class are automatically destroyed. The synthesized destructor has an empty function body.

Destructors are always invoked in the reverse order from which the constructors are run. In our example, the order in which the destructors are called is `~Panda`, `~Endangered`, `~Bear`, `~ZooAnimal`.

Copy and Move Operations for Multiply Derived Classes

As is the case for single inheritance, classes with multiple bases that define their own copy/move constructors and assignment operators must copy, move, or assign the whole object (§ 15.7.2, p. 623). The base parts of a multiply derived class are automatically copied, moved, or assigned only if the derived class uses the synthesized versions of these members. In the synthesized copy-control members, each base class is implicitly constructed, assigned, or destroyed, using the corresponding member from that base class.

For example, assuming that `Panda` uses the synthesized members, then the initialization of `ling_ling`:

```

Panda ying_yang("ying_yang");
Panda ling_ling = ying_yang;    // uses the copy constructor

```

will invoke the `Bear` copy constructor, which in turn runs the `ZooAnimal` copy constructor before executing the `Bear` copy constructor. Once the `Bear` portion of `ling_ling` is constructed, the `Endangered` copy constructor is run to create that part of the object. Finally, the `Panda` copy constructor is run. Similarly, for the synthesized move constructor.

The synthesized copy-assignment operator behaves similarly to the copy constructor. It assigns the `Bear` (and through `Bear`, the `ZooAnimal`) parts of the object first. Next, it assigns the `Endangered` part, and finally the `Panda` part. Move assignment behaves similarly.

18.3.2 Conversions and Multiple Base Classes

Under single inheritance, a pointer or a reference to a derived class can be converted automatically to a pointer or a reference to an accessible base class (§ 15.2.2, p. 597, and § 15.5, p. 613). The same holds true with multiple inheritance. A pointer or reference to any of an object's (accessible) base classes can be used to point or

EXERCISES SECTION 18.3.1

Exercise 18.21: Explain the following declarations. Identify any that are in error and explain why they are incorrect:

- (a) `class CADVehicle : public CAD, Vehicle { ... };`
- (b) `class Dbllist: public List, public List { ... };`
- (c) `class iostream: public istream, public ostream { ... };`

Exercise 18.22: Given the following class hierarchy, in which each class defines a default constructor:

```
class A { ... };
class B : public A { ... };
class C : public B { ... };
class X { ... };
class Y { ... };
class Z : public X, public Y { ... };
class MI : public C, public Z { ... };
```

what is the order of constructor execution for the following definition?

```
MI mi;
```

refer to a derived object. For example, a pointer or reference to `ZooAnimal`, `Bear`, or `Endangered` can be bound to a `Panda` object:

```
// operations that take references to base classes of type Panda
void print(const Bear&);
void highlight(const Endangered&);
ostream& operator<<(ostream&, const ZooAnimal&);
Panda ying_yang("ying_yang");
print(ying_yang);           // passes Panda to a reference to Bear
highlight(ying_yang);      // passes Panda to a reference to Endangered
cout << ying_yang << endl; // passes Panda to a reference to ZooAnimal
```

The compiler makes no attempt to distinguish between base classes in terms of a derived-class conversion. Converting to each base class is equally good. For example, if there was an overloaded version of `print`:

```
void print(const Bear&);
void print(const Endangered&);
```

an unqualified call to `print` with a `Panda` object would be a compile-time error:

```
Panda ying_yang("ying_yang");
print(ying_yang);           // error: ambiguous
```

Lookup Based on Type of Pointer or Reference

As with single inheritance, the static type of the object, pointer, or reference determines which members we can use (§ 15.6, p. 617). If we use a `ZooAnimal` pointer,

only the operations defined in that class are usable. The Bear-specific, Panda-specific, and Endangered portions of the Panda interface are invisible. Similarly, a Bear pointer or reference knows only about the Bear and ZooAnimal members; an Endangered pointer or reference is limited to the Endangered members.

As an example, consider the following calls, which assume that our classes define the virtual functions listed in Table 18.1:

```
Bear *pb = new Panda("ying_yang");
pb->print();           // ok: Panda::print()
pb->cuddle();          // error: not part of the Bear interface
pb->highlight();       // error: not part of the Bear interface
delete pb;             // ok: Panda::~~Panda()
```

When a Panda is used via an Endangered pointer or reference, the Panda-specific and Bear portions of the Panda interface are invisible:

```
Endangered *pe = new Panda("ying_yang");
pe->print();           // ok: Panda::print()
pe->toes();            // error: not part of the Endangered interface
pe->cuddle();          // error: not part of the Endangered interface
pe->highlight();       // ok: Panda::highlight()
delete pe;            // ok: Panda::~~Panda()
```

Table 18.1: Virtual Functions in the ZooAnimal/Endangered Classes

Function	Class Defining Own Version
print	ZooAnimal::ZooAnimal
	Bear::Bear
	Endangered::Endangered
	Panda::Panda
highlight	Endangered::Endangered
	Panda::Panda
toes	Bear::Bear
	Panda::Panda
cuddle	Panda::Panda
destructor	ZooAnimal::ZooAnimal
	Endangered::Endangered

18.3.3 Class Scope under Multiple Inheritance

Under single inheritance, the scope of a derived class is nested within the scope of its direct and indirect base classes (§ 15.6, p. 617). Lookup happens by searching up the inheritance hierarchy until the given name is found. Names defined in a derived class hide uses of that name inside a base.

Under multiple inheritance, this same lookup happens *simultaneously* among all the direct base classes. If a name is found through more than one base class, then use of that name is ambiguous.

EXERCISES SECTION 18.3.2

Exercise 18.23: Using the hierarchy in exercise 18.22 along with class D defined below, and assuming each class defines a default constructor, which, if any, of the following conversions are not permitted?

```
class D : public X, public C { ... };
D *pd = new D;

(a) X *px = pd;      (b) A *pa = pd;
(c) B *pb = pd;      (d) C *pc = pd;
```

Exercise 18.24: On page 807 we presented a series of calls made through a Bear pointer that pointed to a Panda object. Explain each call assuming we used a ZooAnimal pointer pointing to a Panda object instead.

Exercise 18.25: Assume we have two base classes, Base1 and Base2, each of which defines a virtual member named print and a virtual destructor. From these base classes we derive the following classes, each of which redefines the print function:

```
class D1 : public Base1 { /* ... */ };
class D2 : public Base2 { /* ... */ };
class MI : public D1, public D2 { /* ... */ };
```

Using the following pointers, determine which function is used in each call:

```
Base1 *pb1 = new MI;
Base2 *pb2 = new MI;
D1 *pd1 = new MI;
D2 *pd2 = new MI;

(a) pb1->print();  (b) pd1->print();  (c) pd2->print();
(d) delete pb2;   (e) delete pd1;   (f) delete pd2;
```

In our example, if we use a name through a Panda object, pointer, or reference, both the Endangered and the Bear/ZooAnimal subtrees are examined in parallel. If the name is found in more than one subtree, then the use of the name is ambiguous. It is perfectly legal for a class to inherit multiple members with the same name. However, if we want to use that name, we must specify which version we want to use.



WARNING

When a class has multiple base classes, it is possible for that derived class to inherit a member with the same name from two or more of its base classes. Unqualified uses of that name are ambiguous.

For example, if both ZooAnimal and Endangered define a member named max_weight, and Panda does not define that member, this call is an error:

```
double d = ying_yang.max_weight();
```

The derivation of Panda, which results in Panda having two members named max_weight, is perfectly legal. The derivation generates a *potential* ambiguity. That ambiguity is avoided if no Panda object ever calls max_weight. The error

would also be avoided if each call to `max_weight` specifically indicated which version to run—`ZooAnimal::max_weight` or `Endangered::max_weight`. An error results only if there is an ambiguous attempt to use the member.

The ambiguity of the two inherited `max_weight` members is reasonably obvious. It might be more surprising to learn that an error would be generated even if the two inherited functions had different parameter lists. Similarly, it would be an error even if the `max_weight` function were `private` in one class and `public` or `protected` in the other. Finally, if `max_weight` were defined in `Bear` and not in `ZooAnimal`, the call would still be in error.

As always, name lookup happens before type checking (§ 6.4.1, p. 234). When the compiler finds `max_weight` in two different scopes, it generates an error noting that the call is ambiguous.

The best way to avoid potential ambiguities is to define a version of the function in the derived class that resolves the ambiguity. For example, we should give our `Panda` class a `max_weight` function that resolves the ambiguity:

```
double Panda::max_weight() const
{
    return std::max(ZooAnimal::max_weight(),
                   Endangered::max_weight());
}
```

EXERCISES SECTION 18.3.3

Exercise 18.26: Given the hierarchy in the box on page 810, why is the following call to `print` an error? Revise `MI` to allow this call to print to compile and execute correctly.

```
MI mi;
mi.print(42);
```

Exercise 18.27: Given the class hierarchy in the box on page 810 and assuming we add a function named `foo` to `MI` as follows:

```
int ival;
double dval;
void MI::foo(double cval)
{
    int dval;
    // exercise questions occur here
}
```

- List all the names visible from within `MI::foo`.
- Are any names visible from more than one base class?
- Assign to the local instance of `dval` the sum of the `dval` member of `Base1` and the `dval` member of `Derived`.
- Assign the value of the last element in `MI::dvec` to `Base2::fval`.
- Assign `cval` from `Base1` to the first character in `sval` from `Derived`.

CODE FOR EXERCISES TO SECTION 18.3.3

```

struct Base1 {
    void print(int) const;      // public by default
protected:
    int    ival;
    double dval;
    char   cval;
private:
    int    *id;
};

struct Base2 {
    void print(double) const;   // public by default
protected:
    double fval;
private:
    double dval;
};

struct Derived : public Base1 {
    void print(std::string) const; // public by default
protected:
    std::string sval;
    double      dval;
};

struct MI : public Derived, public Base2 {
    void print(std::vector<double>); // public by default
protected:
    int                *ival;
    std::vector<double> dvec;
};

```

18.3.4 Virtual Inheritance

Although the derivation list of a class may not include the same base class more than once, a class can inherit from the same base class more than once. It might inherit the same base indirectly from two of its own direct base classes, or it might inherit a particular class directly and indirectly through another of its base classes.

As an example, the IO library `istream` and `ostream` classes each inherit from a common abstract base class named `basic_ios`. That class holds the stream's buffer and manages the stream's condition state. The class `iostream`, which can both read and write to a stream, inherits directly from both `istream` and `ostream`. Because both types inherit from `basic_ios`, `iostream` inherits that base class twice, once through `istream` and once through `ostream`.

By default, a derived object contains a separate subpart corresponding to each class in its derivation chain. If the same base class appears more than once in the derivation, then the derived object will have more than one subobject of that type.

This default doesn't work for a class such as `iostream`. An `iostream` object

wants to use the same buffer for both reading and writing, and it wants its condition state to reflect both input and output operations. If an `iostream` object has two copies of its `basic_ios` class, this sharing isn't possible.

In C++ we solve this kind of problem by using **virtual inheritance**. Virtual inheritance lets a class specify that it is willing to share its base class. The shared base-class subobject is called a **virtual base class**. Regardless of how often the same virtual base appears in an inheritance hierarchy, the derived object contains only one, shared subobject for that virtual base class.

A Different Panda Class

In the past, there was some debate as to whether panda belongs to the raccoon or the bear family. To reflect this debate, we can change Panda to inherit from both Bear and Raccoon. To avoid giving Panda two ZooAnimal base parts, we'll define Bear and Raccoon to inherit virtually from ZooAnimal. Figure 18.3 illustrates our new hierarchy.

Looking at our new hierarchy, we'll notice a nonintuitive aspect of virtual inheritance. The virtual derivation has to be made before the need for it appears. For example, in our classes, the need for virtual inheritance arises only when we define Panda. However, if Bear and Raccoon had not specified `virtual` on their derivation from ZooAnimal, the designer of the Panda class would be out of luck.

In practice, the requirement that an intermediate base class specify its inheritance as virtual rarely causes any problems. Ordinarily, a class hierarchy that uses virtual inheritance is designed at one time either by one individual or by a single project design group. It is exceedingly rare for a class to be developed independently that needs a virtual base in one of its base classes and in which the developer of the new base class cannot change the existing hierarchy.



Virtual derivation affects the classes that subsequently derive from a class with a virtual base; it doesn't affect the derived class itself.

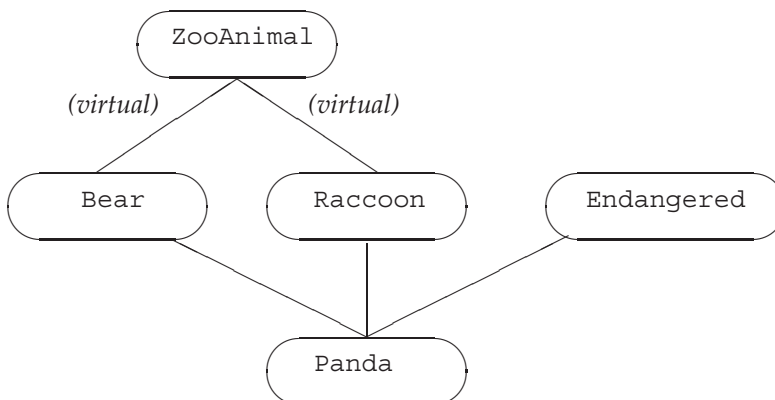


Figure 18.3: Virtual Inheritance Panda Hierarchy

Using a Virtual Base Class

We specify that a base class is virtual by including the keyword `virtual` in the derivation list:

```
// the order of the keywords public and virtual is not significant
class Raccoon : public virtual ZooAnimal { /* ... */ };
class Bear : virtual public ZooAnimal { /* ... */ };
```

Here we've made `ZooAnimal` a virtual base class of both `Bear` and `Raccoon`.

The virtual specifier states a willingness to share a single instance of the named base class within a subsequently derived class. There are no special constraints on a class used as a virtual base class.

We do nothing special to inherit from a class that has a virtual base:

```
class Panda : public Bear,
              public Raccoon, public Endangered {
};
```

Here `Panda` inherits `ZooAnimal` through both its `Raccoon` and `Bear` base classes. However, because those classes inherited virtually from `ZooAnimal`, `Panda` has only one `ZooAnimal` base subpart.

Normal Conversions to Base Are Supported

An object of a derived class can be manipulated (as usual) through a pointer or a reference to an accessible base-class type regardless of whether the base class is virtual. For example, all of the following `Panda` base-class conversions are legal:

```
void dance(const Bear&);
void rummage(const Raccoon&);
ostream& operator<<(ostream&, const ZooAnimal&);
Panda ying_yang;
dance(ying_yang);    // ok: passes Panda object as a Bear
rummage(ying_yang); // ok: passes Panda object as a Raccoon
cout << ying_yang;   // ok: passes Panda object as a ZooAnimal
```

Visibility of Virtual Base-Class Members

Because there is only one shared subobject corresponding to each shared virtual base, members in that base can be accessed directly and unambiguously. Moreover, if a member from the virtual base is overridden along only one derivation path, then that overridden member can still be accessed directly. If the member is overridden by more than one base, then the derived class generally must define its own version as well.

For example, assume class `B` defines a member named `x`; class `D1` inherits virtually from `B` as does class `D2`; and class `D` inherits from `D1` and `D2`. From the scope of `D`, `x` is visible through both of its base classes. If we use `x` through a `D` object, there are three possibilities:

- If `x` is not defined in either `D1` or `D2` it will be resolved as a member in `B`; there is no ambiguity. A `D` object contains only one instance of `x`.

- If *x* is a member of *B* and also a member in one, but not both, of *D1* and *D2*, there is again no ambiguity—the version in the derived class is given precedence over the shared virtual base class, *B*.
- If *x* is defined in both *D1* and *D2*, then direct access to that member is ambiguous.

As in a nonvirtual multiple inheritance hierarchy, ambiguities of this sort are best resolved by the derived class providing its own instance of that member.

EXERCISES SECTION 18.3.4

Exercise 18.28: Given the following class hierarchy, which inherited members can be accessed without qualification from within the *VMI* class? Which require qualification? Explain your reasoning.

```
struct Base {
    void bar(int); // public by default
protected:
    int ival;
};
struct Derived1 : virtual public Base {
    void bar(char); // public by default
    void foo(char);
protected:
    char cval;
};
struct Derived2 : virtual public Base {
    void foo(int); // public by default
protected:
    int ival;
    char cval;
};
class VMI : public Derived1, public Derived2 { };
```

18.3.5 Constructors and Virtual Inheritance

In a virtual derivation, the virtual base is initialized by the *most derived constructor*. In our example, when we create a *Panda* object, the *Panda* constructor alone controls how the *ZooAnimal* base class is initialized.

To understand this rule, consider what would happen if normal initialization rules applied. In that case, a virtual base class might be initialized more than once. It would be initialized along each inheritance path that contains that virtual base. In our *ZooAnimal* example, if normal initialization rules applied, both *Bear* and *Raccoon* would initialize the *ZooAnimal* part of a *Panda* object.

Of course, each class in the hierarchy might at some point be the “most derived” object. As long as we can create independent objects of a type derived from

a virtual base, the constructors in that class must initialize its virtual base. For example, in our hierarchy, when a `Bear` (or a `Raccoon`) object is created, there is no further derived type involved. In this case, the `Bear` (or `Raccoon`) constructors directly initialize their `ZooAnimal` base as usual:

```
Bear::Bear(std::string name, bool onExhibit):
    ZooAnimal(name, onExhibit, "Bear") { }
Raccoon::Raccoon(std::string name, bool onExhibit)
    : ZooAnimal(name, onExhibit, "Raccoon") { }
```

When a `Panda` is created, it is the most derived type and controls initialization of the shared `ZooAnimal` base. Even though `ZooAnimal` is not a direct base of `Panda`, the `Panda` constructor initializes `ZooAnimal`:

```
Panda::Panda(std::string name, bool onExhibit)
    : ZooAnimal(name, onExhibit, "Panda"),
      Bear(name, onExhibit),
      Raccoon(name, onExhibit),
      Endangered(Endangered::critical),
      sleeping_flag(false) { }
```

How a Virtually Inherited Object Is Constructed

The construction order for an object with a virtual base is slightly modified from the normal order: The virtual base subparts of the object are initialized first, using initializers provided in the constructor for the most derived class. Once the virtual base subparts of the object are constructed, the direct base subparts are constructed in the order in which they appear in the derivation list.

For example, when a `Panda` object is created:

- The (virtual base class) `ZooAnimal` part is constructed first, using the initializers specified in the `Panda` constructor initializer list.
- The `Bear` part is constructed next.
- The `Raccoon` part is constructed next.
- The third direct base, `Endangered`, is constructed next.
- Finally, the `Panda` part is constructed.

If the `Panda` constructor does not explicitly initialize the `ZooAnimal` base class, then the `ZooAnimal` default constructor is used. If `ZooAnimal` doesn't have a default constructor, then the code is in error.



Virtual base classes are always constructed prior to nonvirtual base classes regardless of where they appear in the inheritance hierarchy.

Constructor and Destructor Order

A class can have more than one virtual base class. In that case, the virtual subobjects are constructed in left-to-right order as they appear in the derivation list. For

example, in the following whimsical `TeddyBear` derivation, there are two virtual base classes: `ToyAnimal`, a direct virtual base, and `ZooAnimal`, which is a virtual base class of `Bear`:

```
class Character { /* ... */ };
class BookCharacter : public Character { /* ... */ };
class ToyAnimal { /* ... */ };
class TeddyBear : public BookCharacter,
                  public Bear, public virtual ToyAnimal
                  { /* ... */ };
```

The direct base classes are examined in declaration order to determine whether there are any virtual base classes. If so, the virtual bases are constructed first, followed by the nonvirtual base-class constructors in declaration order. Thus, to create a `TeddyBear`, the constructors are invoked in the following order:

```
ZooAnimal();           // Bear's virtual base class
ToyAnimal();           // direct virtual base class
Character();           // indirect base class of first nonvirtual base class
BookCharacter();       // first direct nonvirtual base class
Bear();                // second direct nonvirtual base class
TeddyBear();           // most derived class
```

The same order is used in the synthesized copy and move constructors, and members are assigned in this order in the synthesized assignment operators. As usual, an object is destroyed in reverse order from which it was constructed. The `TeddyBear` part will be destroyed first and the `ZooAnimal` part last.

EXERCISES SECTION 18.3.5

Exercise 18.29: Given the following class hierarchy:

```
class Class { ... };
class Base : public Class { ... };
class D1 : virtual public Base { ... };
class D2 : virtual public Base { ... };
class MI : public D1, public D2 { ... };
class Final : public MI, public Class { ... };
```

- In what order are constructors and destructors run on a `Final` object?
- A `Final` object has how many `Base` parts? How many `Class` parts?
- Which of the following assignments is a compile-time error?

```
Base *pb;    Class *pc;    MI *pmi;    D2 *pd2;
```

- `pb = new Class;`
- `pc = new Final;`
- `pmi = pb;`
- `pd2 = pmi;`

Exercise 18.30: Define a default constructor, a copy constructor, and a constructor that has an `int` parameter in `Base`. Define the same three constructors in each derived class. Each constructor should use its argument to initialize its `Base` part.

CHAPTER SUMMARY

C++ is used to solve a wide range of problems—from those solvable in a few hours' time to those that take years of development by large teams. Some features in C++ are most applicable in the context of large-scale problems: exception handling, namespaces, and multiple or virtual inheritance.

Exception handling lets us separate the error-detection part of the program from the error-handling part. When an exception is thrown, the current executing function is suspended and a search is started to find the nearest matching `catch` clause. Local variables defined inside functions that are exited while searching for a `catch` clause are destroyed as part of handling the exception.

Namespaces are a mechanism for managing large, complicated applications built from code produced by independent suppliers. A namespace is a scope in which objects, types, functions, templates, and other namespaces may be defined. The standard library is defined inside the namespace named `std`.

Conceptually, multiple inheritance is a simple notion: A derived class may inherit from more than one direct base class. The derived object consists of the derived part and a base part contributed by each of its base classes. Although conceptually simple, the details can be more complicated. In particular, inheriting from multiple base classes introduces new possibilities for name collisions and resulting ambiguous references to names from the base part of an object.

When a class inherits directly from more than one base class, it is possible that those classes may themselves share another base class. In such cases, the intermediate classes can opt to make their inheritance virtual, which states a willingness to share their virtual base class with other classes in the hierarchy that inherit virtually from that same base class. In this way there is only one copy of the shared virtual base in a subsequently derived class.

DEFINED TERMS

catch-all A `catch` clause in which the exception declaration is `(...)`. A catch-all clause catches an exception of any type. It is typically used to catch an exception that is detected locally in order to do local cleanup. The exception is then rethrown to another part of the program to deal with the underlying cause of the problem.

catch clause Part of the program that handles an exception. A `catch` clause consists of the keyword `catch` followed by an exception declaration and a block of statements. The code inside a `catch` does whatever is necessary to handle an exception of the type defined in its exception declaration.

constructor order Under nonvirtual inheritance, base classes are constructed in the order in which they are named in the class derivation list. Under virtual inheritance, the virtual base class(es) are constructed before any other bases. They are constructed in the order in which they appear in the derivation list of the derived type. Only the most derived type may initialize a virtual base; constructor initializers for that base that appear in the intermediate base classes are ignored.

exception declaration `catch` clause declaration that specifies the type of exception that the `catch` can handle. The declaration acts like a parameter list, whose single parameter is initialized by the exception

object. If the exception specifier is a non-reference type, then the exception object is copied to the `catch`.

exception handling Language-level support for managing run-time anomalies. One independently developed section of code can detect and “raise” an exception that another independently developed part of the program can “handle.” The error-detecting part of the program throws an exception; the error-handling part handles the exception in a `catch` clause of a `try` block.

exception object Object used to communicate between the `throw` and `catch` sides of an exception. The object is created at the point of the `throw` and is a copy of the thrown expression. The exception object exists until the last handler for the exception completes. The type of the object is the static type of the thrown expression.

file static Name local to a file that is declared with the `static` keyword. In C and pre-Standard versions of C++, file statics were used to declare objects that could be used in a single file only. File statics are deprecated in C++, having been superseded by the use of unnamed namespaces.

function try block Used to catch exceptions from a constructor initializer. The keyword `try` appears before the colon that starts the constructor initializer list (or before the open curly of the constructor body if the initializer list is empty) and closes with one or more `catch` clauses that appear after the close curly of the constructor body.

global namespace The (implicit) namespace in each program that holds all global definitions.

handler Synonym for a `catch` clause.

inline namespace Members of a namespace designated as `inline` can be used as if they were members of an enclosing namespace.

multiple inheritance Class with more than one direct base class. The derived class inherits the members of all its base classes. A

separate access specifier may be provided for each base class.

namespace Mechanism for gathering all the names defined by a library or other collection of programs into a single scope. Unlike other scopes in C++, a namespace scope may be defined in several parts. The namespace may be opened and closed and reopened again in disparate parts of the program.

namespace alias Mechanism for defining a synonym for a given namespace:

```
namespace N1 = N;
```

defines `N1` as another name for the namespace named `N`. A namespace can have multiple aliases; the namespace name or any of its aliases may be used interchangeably.

namespace pollution Occurs when all the names of classes and functions are placed in the global namespace. Large programs that use code written by multiple independent parties often encounter collisions among names if these names are global.

noexcept operator Operator that returns a `bool` indicating whether a given expression might throw an exception. The expression is unevaluated. The result is a constant expression. Its value is `true` if the expression does not contain a `throw` and calls only functions designated as `nonthrowing`; otherwise the result is `false`.

noexcept specification Keyword used to indicate whether a function throws. When `noexcept` follows a function’s parameter list, it may be optionally followed by a parenthesized constant expression that must be convertible to `bool`. If the expression is omitted, or if it is `true`, the function throws no exceptions. An expression that is `false` or a function that has no exception specification may throw any exception.

nonthrowing specification An exception specification that promises that a function won't throw. If a nonthrowing function does throw, `terminate` is called. Nonthrowing specifiers are `noexcept` without an argument or with an argument that evaluates as `true` and `throw()`.

raise Often used as a synonym for `throw`. C++ programmers speak of "throwing" or "raising" an exception interchangeably.

rethrow A `throw` that does not specify an expression. A `rethrow` is valid only from inside a `catch` clause, or in a function called directly or indirectly from a `catch`. Its effect is to rethrow the exception object that it received.

stack unwinding The process whereby the functions are exited in the search for a `catch`. Local objects constructed before the exception are destroyed before entering the corresponding `catch`.

terminate Library function that is called if an exception is not caught or if an exception occurs while a handler is in process. `terminate` ends the program.

throw e Expression that interrupts the current execution path. Each `throw` transfers control to the nearest enclosing `catch` clause that can handle the type of exception that is thrown. The expression `e` is copied into the exception object.

try block Block of statements enclosed by the keyword `try` and one or more `catch` clauses. If the code inside the `try` block raises an exception and one of the `catch` clauses matches the type of the exception, then the exception is handled by that `catch`. Otherwise, the exception is passed out of the `try` to a `catch` further up the call chain.

unnamed namespace Namespace that is defined without a name. Names defined in an unnamed namespace may be accessed directly without use of the scope operator. Each file has its own unique unnamed namespace. Names in an unnamed namespace are not visible outside that file.

using declaration Mechanism to inject a single name from a namespace into the current scope:

```
using std::cout;
```

makes the name `cout` from the namespace `std` available in the current scope. The name `cout` can subsequently be used without the `std::` qualifier.

using directive Declaration of the form

```
using NS;
```

makes *all* the names in the namespace named `NS` available in the nearest scope containing both the `using` directive and the namespace itself.

virtual base class Base class that specifies `virtual` in its own derivation list. A virtual base part occurs only once in a derived object even if the same class appears as a virtual base more than once in the hierarchy. In nonvirtual inheritance a constructor may initialize only its direct base class(es). When a class is inherited virtually, that class is initialized by the most derived class, which therefore should include an initializer for all of its virtual parent(s).

virtual inheritance Form of multiple inheritance in which derived classes share a single copy of a base that is included in the hierarchy more than once.

:: operator Scope operator. Used to access names from a namespace or a class.